

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО
ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В.
Г. ШУХОВА» (БГТУ им. В.Г. Шухова)**

Кафедра программного обеспечения вычислительной техники и
автоматизированных систем

Лабораторная работа №1
по дисциплине: Основы ИИ
тема: «Алгоритм отжига»

Выполнил: ст. группы ПВ-223
Дмитриев Андрей Александрович

Проверил:
Твердохлеб Виталий Викторович

Белгород 2025 г.

Цель работы: Разработка и исследование алгоритма отжига в процессе решения числовой задачи оптимизации.

Задание к лабораторной работе:

1. Изучить теоретическую часть лекционного материала по теме. Ответить на контрольные вопросы.
2. Используя пример программного кода (исходные файлы в приложении к заданию), а также любой высокоуровневый язык программирования и соответствующую среду, разработать программное обеспечение с графическим интерфейсом, позволяющее решить задачу N-ферзей для $N > 20$. Интерфейс ПО должен обеспечивать ввод следующих параметров алгоритма:
 - Максимальная температура
 - Минимальная температура
 - Коэффициент понижения температуры
 - Количество ферзей.
 - Количество шагов при постоянном значении температуры.

Необходимо предусмотреть визуализацию лучшего решения в виде шахматной доски, а также необходимо построить график изменения принятых плохих решений, энергии лучшего решения, и температуры.

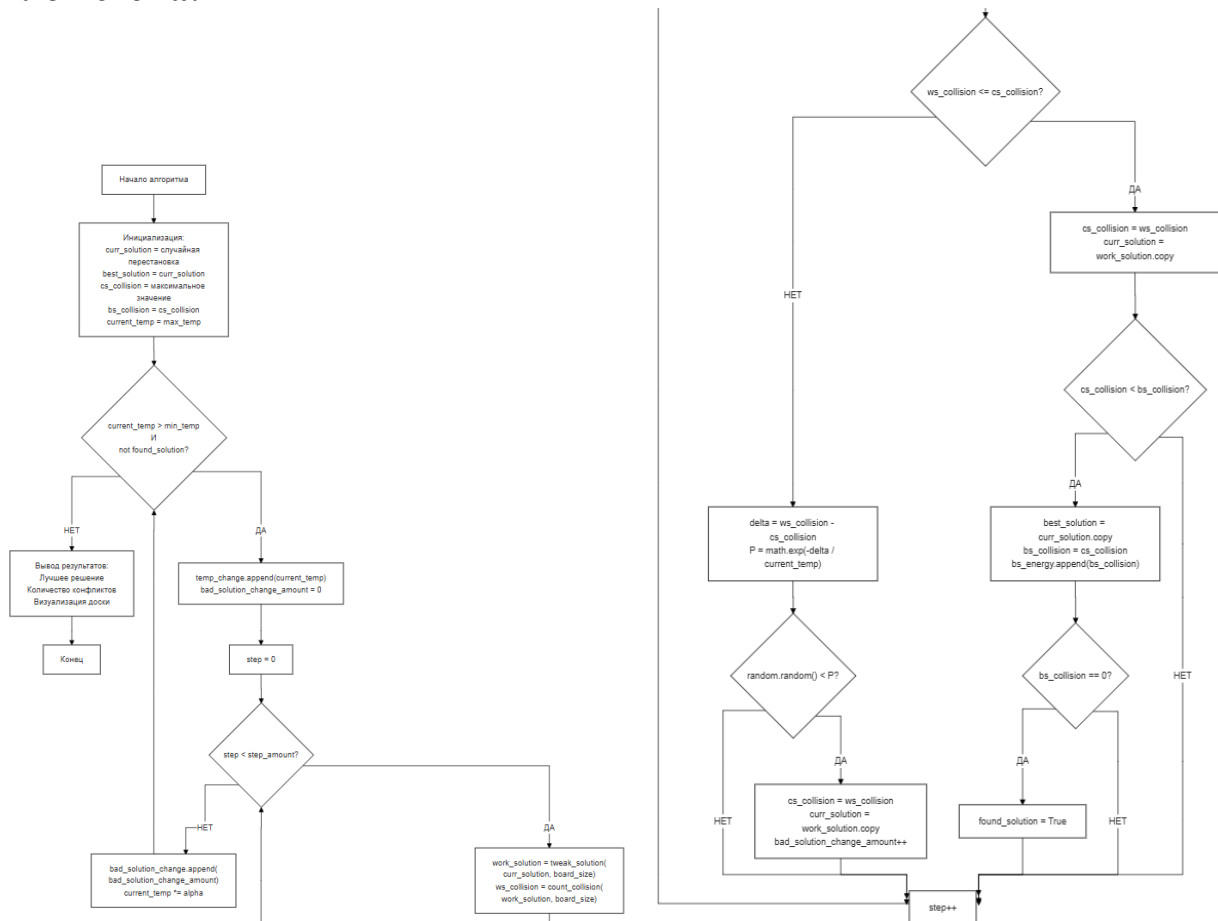
3. Исследовать влияние параметров алгоритма на качество его работы, провести не менее 20 экспериментов при разных комбинациях параметров.

Содержание отчёта:

1. Наименование и цель работы.
2. Блок-схема алгоритма решения задачи N-ферзей с использованием метода обратного восстановления.
3. Исходный код модулей программного обеспечения.
4. Результаты исследования влияния параметров алгоритма на его эффективность, оформленные в виде таблицы.
4. Выводы по работе.

Ход работы:

Блок схема:



Исходный код:

```
import random
import math
from typing import List
import matplotlib.pyplot as plt
import numpy as np

def tweak_solution(
    solution: List[int],
    board_size: int
) -> List[int]:
    """Модифицирует текущее"""
    new_solution = solution.copy()
    col = random.randint(0, board_size - 1) # Выбираем случайный столбец

    # Генерируем новую позицию, отличную от текущей
    current_row = solution[col]
    possible_rows = [r for r in range(board_size) if r != current_row]

    if possible_rows:
        new_solution[col] = random.choice(possible_rows)

    return new_solution

def count_collision(solution: List[int], board_size: int) -> int:
    """Подсчитывает количество конфликтов (атак)"""
    collision = 0
    for i in range(board_size):
        for j in range(board_size):
            # Проверка на диагональные атаки и атаки по вертикали
```

```

        if abs(i - j) == abs(solution[i] - solution[j]) or solution[i]
== solution[j]:
            collision += 1
        return collision

def visualize_board(
    best_solution: List[int],

    board_size: int
): """Визуализирует шахматную доску с ферзями"""

def log_info(
    file_name: str,
    bs_energy: List[int],
    bad_solution_change: List[int],
    temp_change: List[float]
): """Сохраняет данные в файл"""

def start_learning(board_size: int, min_temp: float, max_temp: float, alpha:
float, step_amount: int):
    """Запускает процесс обучения алгоритма отжига"""
    # Инициализация решений
    curr_solution = list(range(board_size))
    random.shuffle(curr_solution)
    best_solution = curr_solution.copy()

    # Подсчет коллизий
    cs_collision = (board_size - 1) * board_size
    bs_collision = cs_collision

    # Логирование
    bs_energy = [bs_collision]
    bad_solution_change = []
    temp_change = []

    current_temp = max_temp
    found_solution = False

    while current_temp > min_temp and not found_solution:
        temp_change.append(current_temp)
        bad_solution_change_amount = 0

        for step in range(step_amount):
            # Модифицируем решение
            work_solution = tweak_solution(curr_solution, board_size)
            ws_collision = count_collision(work_solution, board_size)

            if ws_collision <= cs_collision:
                cs_collision = ws_collision
                curr_solution = work_solution.copy()

            if cs_collision < bs_collision:
                best_solution = curr_solution.copy()
                bs_collision = cs_collision
                bs_energy.append(bs_collision)

                if bs_collision == 0:
                    found_solution = True
                    break

            else:
                delta = ws_collision - cs_collision
                P = math.exp(-delta / current_temp)

                if random.random() < P:

```

[illegible]

```

        plot_and_save_array(array=bad, filename="bad.png",
                             title="График количества принятых плохих
решений на каждой итерации",
                             color="red",
                             xlabel="Номер итерации",
                             ylabel="Количество принятых плохих решений")

        plot_and_save_array(array=temp, filename="temp.png",
                             title="График изменения температуры",
                             color="orange",
                             xlabel="Номер итерации",
                             ylabel="Температура")

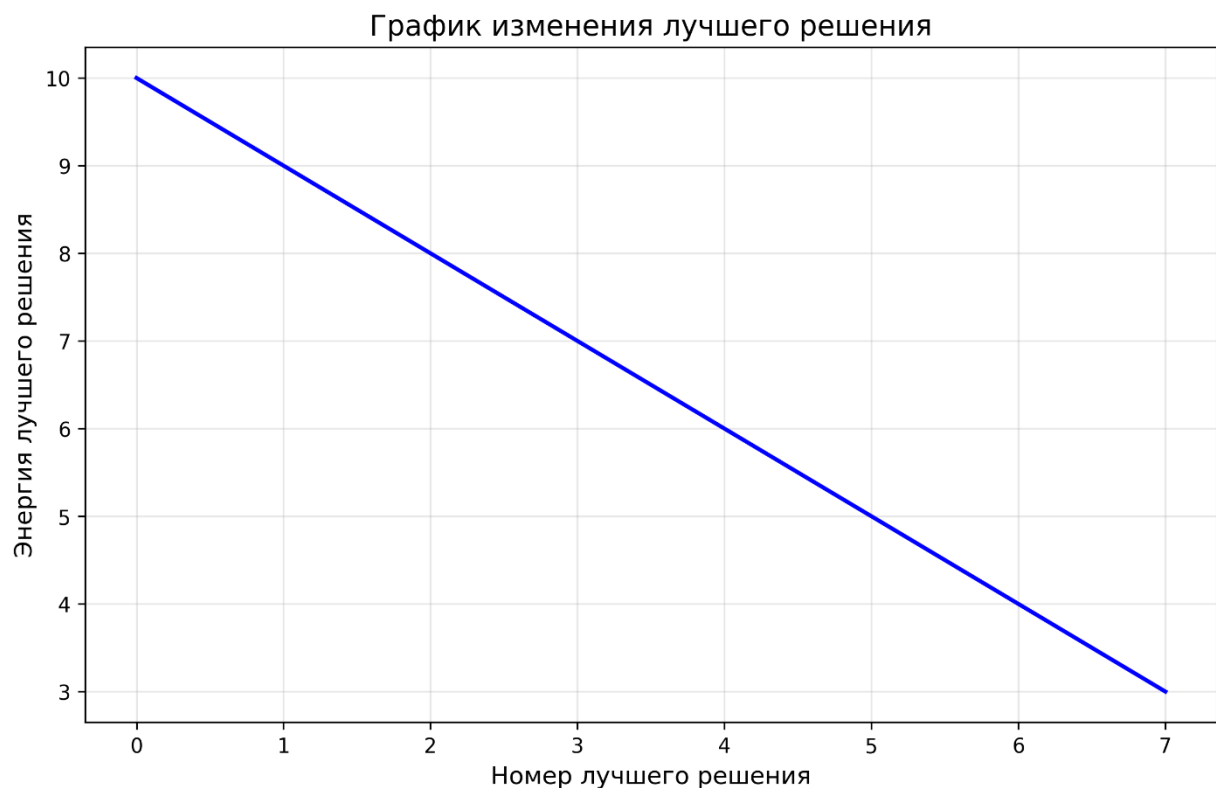
        print("Все графики успешно созданы!")

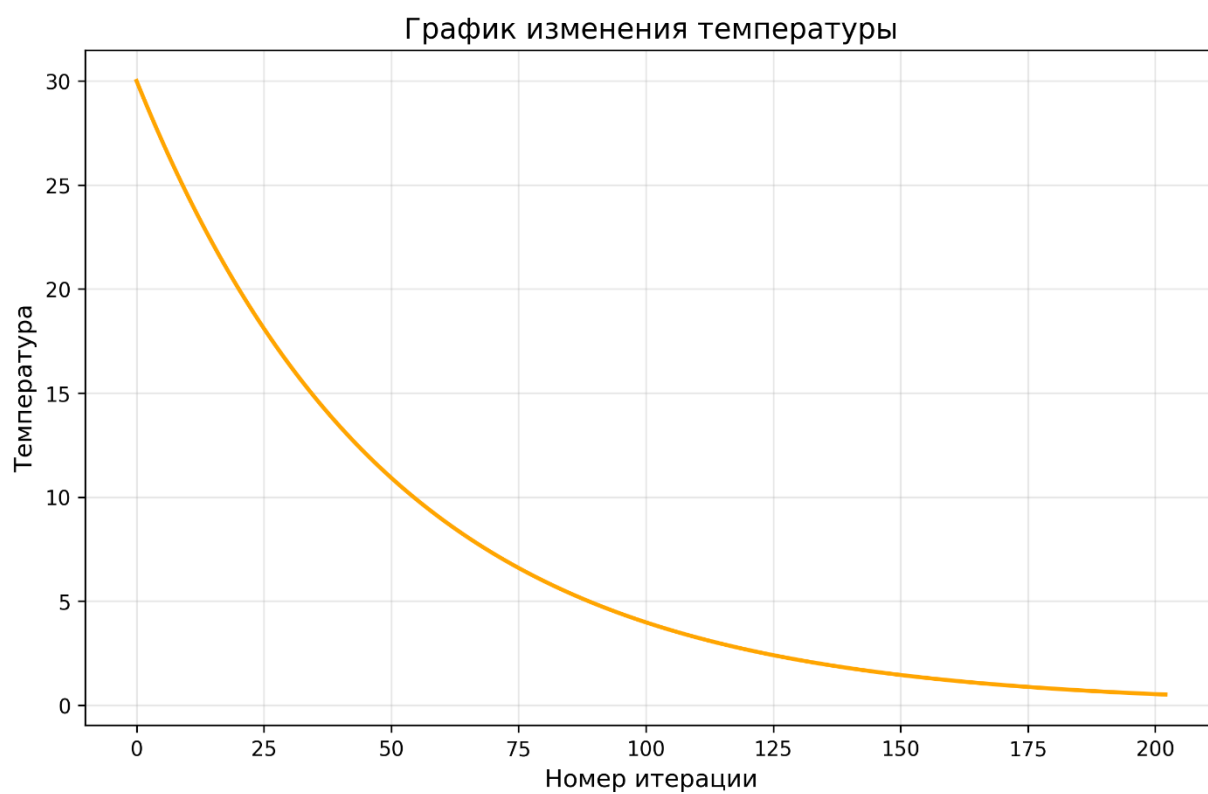
    except FileNotFoundError:
        print("Файл log.txt не найден!")
    except ValueError as e:
        print(f"Ошибка парсинга данных: {e}")
    except Exception as e:
        print(f"Произошла ошибка: {e}")

```

Пример работы программы:

Графики работы программы при: board_size=20, min_temp=0.5, max_temp=30, alpha=0,98, step_amount=1000



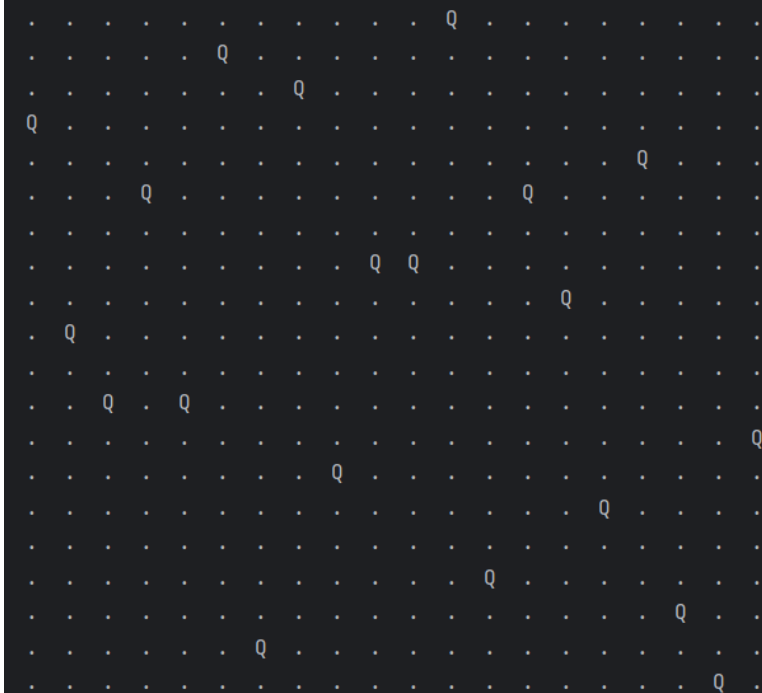


Вывод программы:

Лучшее найденное решение: [4, 10, 12, 6, 12, 2, 19, 3, 14, 8, 8, 1, 17, 6, 9, 15, 5, 18, 20, 13]

Количество конфликтов: 3

Шахматная доска:



Эксперименты:

BOARD SIZE	MIN TEMP	MAX TEMP	ALPHA	STEP AMOUNT
21	0,5	30	0,95	1000
22	0,3	35	0,92	1200
24	0,4	32	0,94	1100
28	0,5	33	0,93	1050
32	0,7	25	0,96	900
38	0,7	28	0,96	950
25	0,7	27	0,97	850
23	0,6	28	0,96	900
26	0,2	40	0,9	1300
27	0,8	26	0,98	800
29	0,6	29	0,95	950
30	0,4	31	0,94	1150
31	0,3	36	0,91	1250
33	0,5	34	0,92	1100
34	0,2	38	0,89	1400
35	0,8	24	0,97	850
36	0,6	30	0,95	1000
37	0,4	35	0,93	1200
39	0,3	37	0,9	1300
40	0,5	33	0,94	1100

Графики работы программы при: board_size=40, min_temp=0.5, max_temp=30, alpha=0,98, step_amount=1000 (конфликты: 9)

График изменения лучшего решения

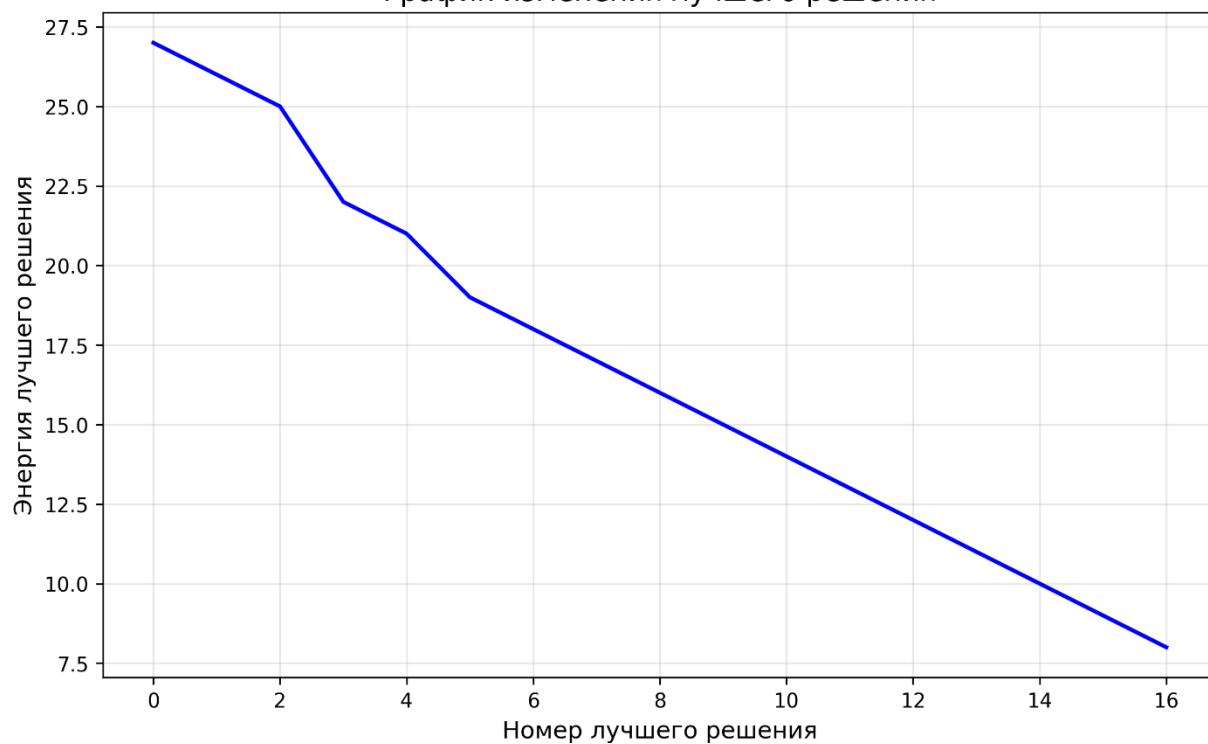
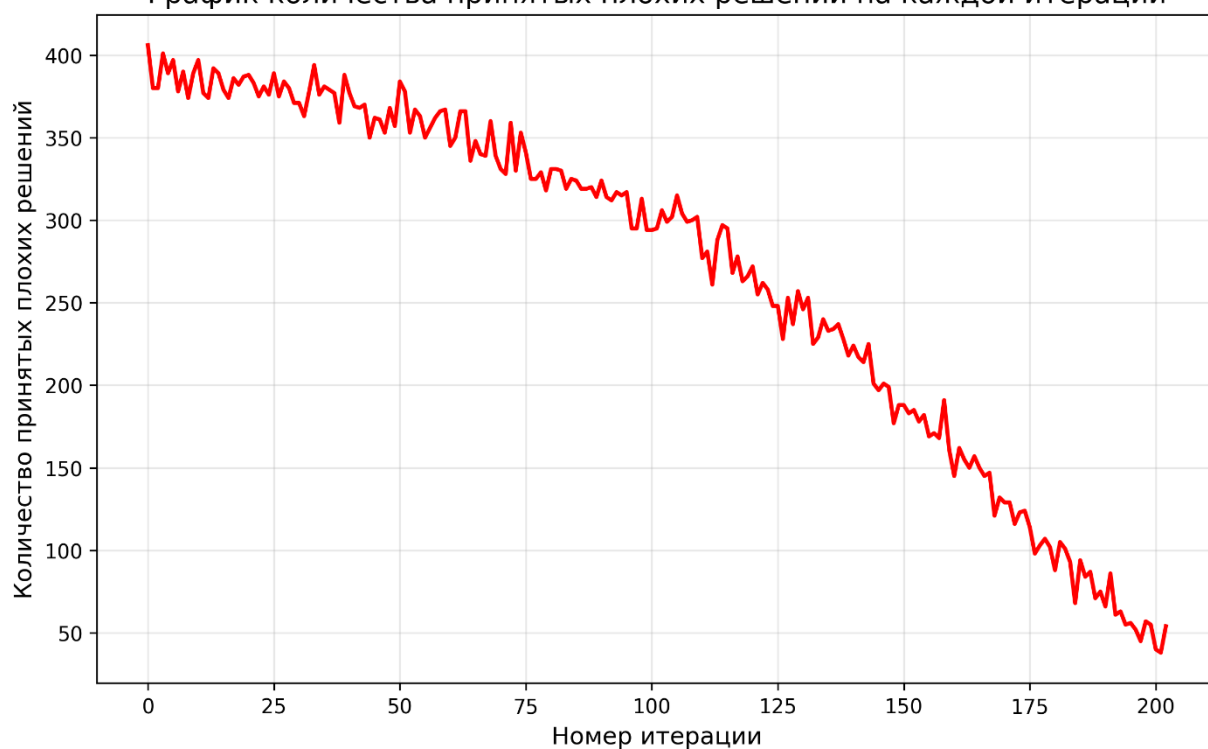


График количества принятых плохих решений на каждой итерации



Графики работы программы при: board_size=20, min_temp=0.5, max_temp=30, alpha=0,999, step_amount=1000 (конфликты: 1)

График изменения лучшего решения

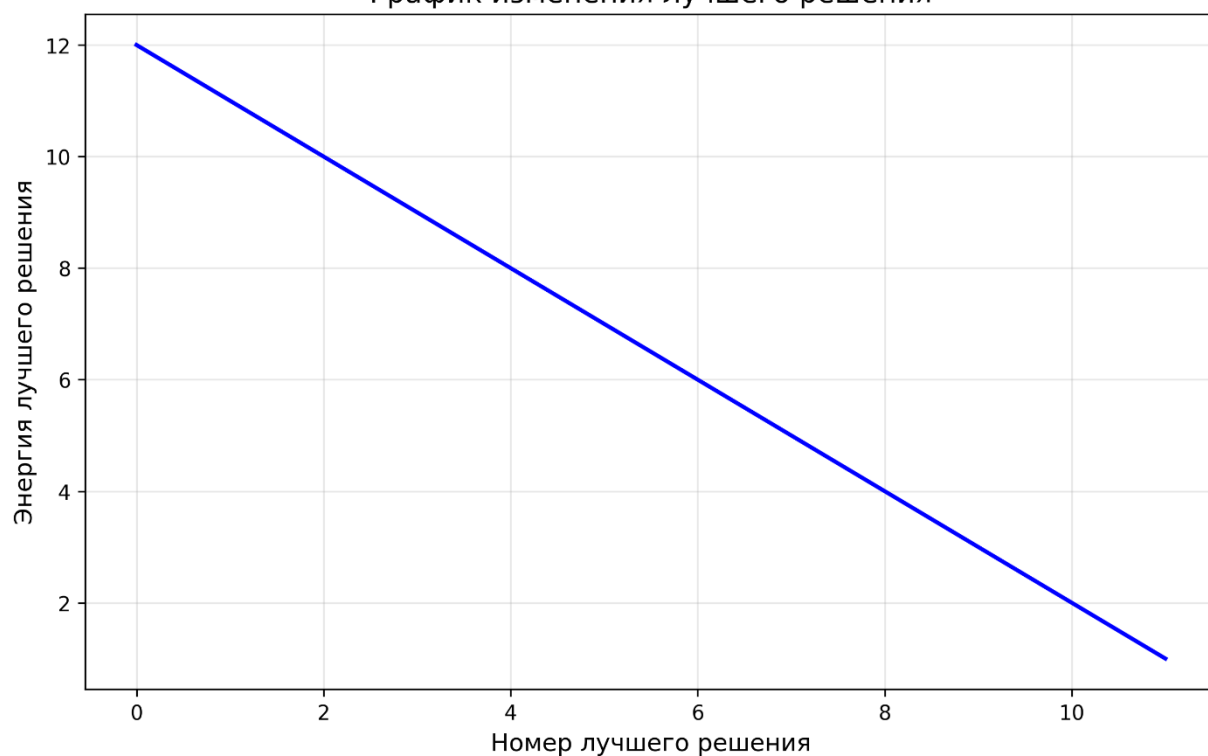
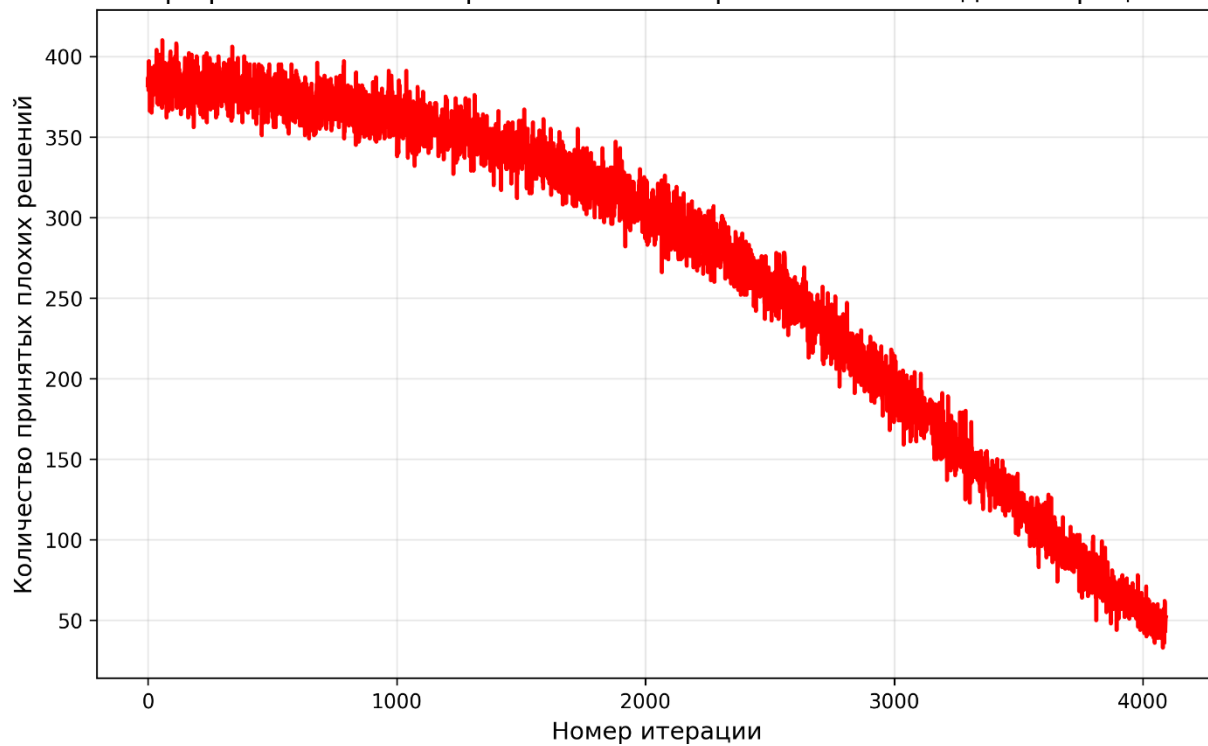
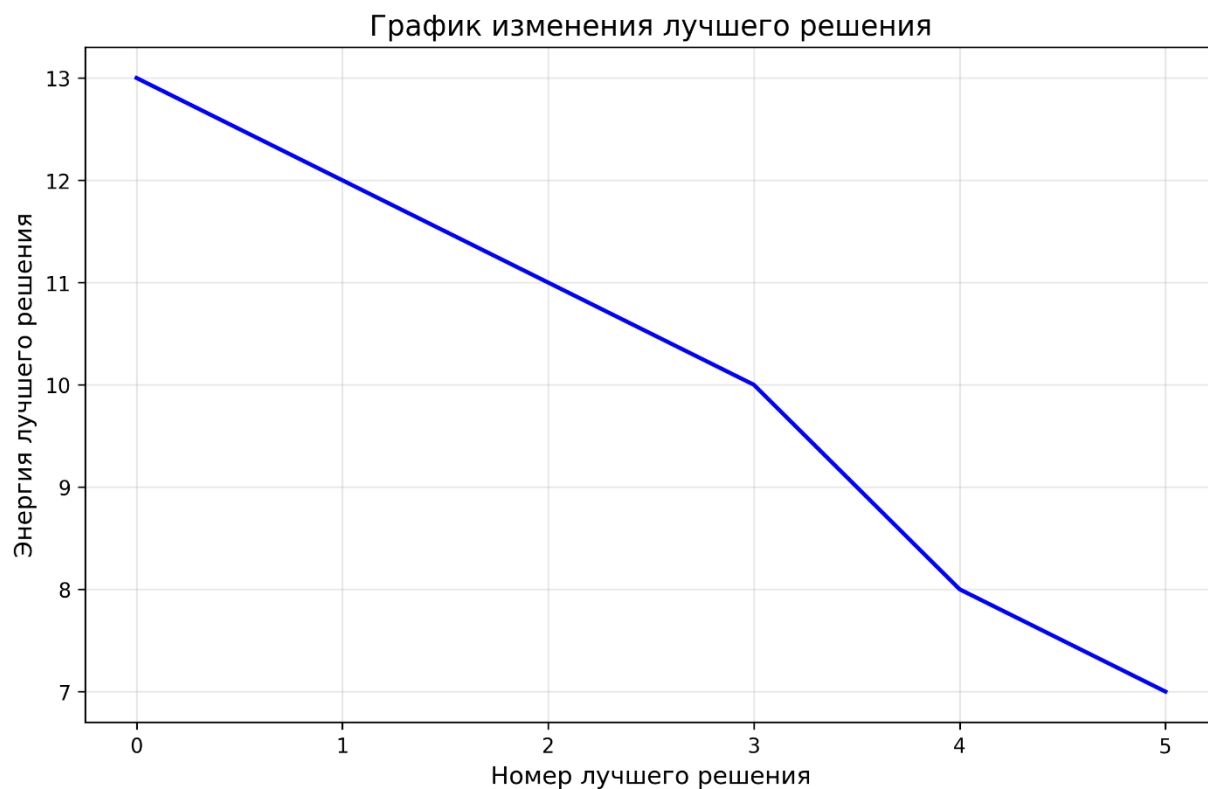


График количества принятых плохих решений на каждой итерации



Графики работы программы при: board_size=20, min_temp=10.0, max_temp=25, alpha=0,999, step_amount=1000 (конфликты: 7)



Вывод: В ходе лабораторной работы разработан и исследован алгоритм отжига в процессе решения числовой задачи оптимизации. Проведены исследования, в ходе которых выяснены оптимальные значения для получения результата наименьшим количеством коллизий: минимальная температура – до 1 максимальная температура – оптимально 30-50, скорость охлаждения – α – 0.95-0.99, количество шагов на температуре – 1000-5000.